COT4400 Project 2

- Yasemin Tuncer
- Westley Yarlott

Section 1: Introduction

The purpose of this project was to understand and apply 3 common algorithmic paradigms in workplace-related environments. We will analyze their efficiency and compare the theoretical vs practical performance. The first program is (D&Q here). The second program is a Server Maintenence Window Planner that employs the Greedy algorithm to pick the best windows in the most efficient manner. The third program is (Knapsack here). The theme for each program was cybersecurity related tasks. The project was accomplished by two members, one member programmed the Server Maintenence program and the main.py file that tests every program. The other member produced the Vulnerability Severity Ranker and the Firewal Rule Budget.

Section 2: Divide & Conquer — Merge Sort (Vulnerability Severity Ranker)

Section 3: Greedy — Activity Selection (Server Maintenance Window Planner)

Problem Explanation

Frequently in real-world settings, server maintenance is needed, but companies like to avoid their website being “down”. Usually companies have a small maintenance window per-night, say after U.S. markets are closed at 1 AM and before European markets open at 4 AM. With things like OS patches, kernel upgrades, firewall rule deployments, you can only run one disruptive task at a time because they all touch the same server. This is where the program comes in. Given all the proposed windows competing for the night’s downtime, which subset should you schedule to maximize tasks completed?

Approach

First, we sort the windows by finish time. Then, the greedy algorithm sweeps through and picks compatible activities. The beauty of the algorithm is that it’s only making a decision based on a single variable: is the start time past the window, the activity is not included in the final list.

Pseudocode

Sort by finish time ascending

```
Start with the first activity (quickest)
For activity in activities
    If start >= last_finish;
        Add the activity to the selection
        Compute the new finish time for next iteration
```

Why Greedy?

The algorithm guarantees that the maximum number of activities will be squeezed into the window. The part of the algorithm where greedy is being used, after it's been sorted, will This is separate from the stage that sorts the list in the first place.

Complexity Analysis

It's easy to think the time complexity is O(n) since all greedy does is pass through the whole list once, but there are actually two stages to this algorithm. The list needs to be sorted in the first place, and we use our merge-sort program for that. The merge-sort takes O(n log n) time. The final complexity comes out to: O(n + n log n) which ends as O(n log n). Space complexity is O(n) because it just needs to store the values held in the array, no additional space is needed for recursion.

Section 4: Dynamic Programming — 0/1 Knapsack (Firewall Rule Budget)

Section 5: Results

Test Cases

Merge sort:

```python
# Each vulnerability: (name, cvss_score)
test_cases = {
    "Small / Random": [
        ("CVE-2025-1001", 7.5),
        ("CVE-2025-1002", 9.8),
        ("CVE-2025-1003", 4.3),
        ("CVE-2025-1004", 6.1),
        ("CVE-2025-1005", 10.0),
    ],
    "Already sorted (descending)": [
        ("CVE-2025-2001", 10.0),
        ("CVE-2025-2002", 8.5),
        ("CVE-2025-2003", 7.0),
        ("CVE-2025-2004", 5.5),
        ("CVE-2025-2005", 3.0),
    ],
    "Reverse sorted (ascending)": [
        ("CVE-2025-3001", 1.0),
        ("CVE-2025-3002", 3.5),
        ("CVE-2025-3003", 5.0),
        ("CVE-2025-3004", 7.5),
        ("CVE-2025-3005", 9.9),
    ],
    "Edge — empty list": [],
    "Edge — single element": [
        ("CVE-2025-4001", 6.0),
    ],
}
```

Activity Selector:

```python
# Each window: (name, start, finish)
test_cases = {
    "Small / Normal": [
        ("Patch-A", 0, 2),
        ("Patch-B", 1, 3),
        ("Patch-C", 2, 5),
        ("Patch-D", 4, 6),
        ("Patch-E", 5, 7),
        ("Patch-F", 6, 8),
    ],
    "All overlapping": [
        ("Patch-X1", 0, 10),
        ("Patch-X2", 1, 9),
        ("Patch-X3", 2, 8),
    ],
    "No overlaps": [
        ("Patch-Y1", 0, 1),
        ("Patch-Y2", 2, 3),
        ("Patch-Y3", 4, 5),
        ("Patch-Y4", 6, 7),
    ],
    "Edge - empty list": [],
    "Edge - single window": [
        ("Patch-Z1", 3, 5),
    ],
}
```

Knapsack:

```python
# Each rule: (name, cpu_cost, coverage_value)
test_cases = {
    "Small / Normal": {
        "rules": [
            ("Block-SSH-Brute", 3, 60),
            ("DPI-Malware", 4, 80),
            ("Rate-Limit-DNS", 2, 40),
            ("Geo-Block", 5, 90),
            ("IDS-Signature", 1, 30),
        ],
        "capacity": 8,
    },
    "Edge — zero capacity": {
        "rules": [
            ("Rule-A", 2, 50),
            ("Rule-B", 3, 70),
        ],
        "capacity": 0,
    },
    "Edge — single item fits exactly": {
        "rules": [
            ("Rule-Solo", 5, 100),
        ],
        "capacity": 5,
    },
    "Edge — all items too heavy": {
        "rules": [
            ("Heavy-1", 10, 50),
            ("Heavy-2", 12, 60),
        ],
        "capacity": 5,
    },
    "Medium": {
        "rules": [
            ("R1", 2, 30), ("R2", 3, 45), ("R3", 4, 50),
            ("R4", 5, 70), ("R5", 1, 20), ("R6", 3, 55),
            ("R7", 6, 85), ("R8", 2, 35), ("R9", 4, 60),
            ("R10", 7, 100),
        ],
        "capacity": 15,
    },
}
```

- **5.2 Experimental Evaluation** — input-size-vs-time table for each algorithm (from your benchmark output)

Tester Program Output

```
============================================================
MERGE SORT — Vulnerability Severity Ranker
============================================================

Test: Small / Random
  Input scores:  [7.5, 9.8, 4.3, 6.1, 10.0]
  Sorted (desc): [10.0, 9.8, 7.5, 6.1, 4.3]
  Time: 0.0073 ms

Test: Already sorted (descending)
  Input scores:  [10.0, 8.5, 7.0, 5.5, 3.0]
  Sorted (desc): [10.0, 8.5, 7.0, 5.5, 3.0]
  Time: 0.0033 ms

Test: Reverse sorted (ascending)
  Input scores:  [1.0, 3.5, 5.0, 7.5, 9.9]
  Sorted (desc): [9.9, 7.5, 5.0, 3.5, 1.0]
  Time: 0.0026 ms

Test: Edge — empty list
  Input scores:  []
  Sorted (desc): []
  Time: 0.0002 ms

Test: Edge — single element
  Input scores:  [6.0]
  Sorted (desc): [6.0]
  Time: 0.0001 ms

--- Benchmark ---
Size       Time (ms)
10         0.0065
100        0.0955
1000       1.2538
5000       7.2764
10000      15.3238
```

```
============================================================
ACTIVITY SELECTION — Server Maintenance Window Planner
============================================================

Test: Small / Normal
  Windows:  [('Patch-A', 0, 2), ('Patch-B', 1, 3), ('Patch-C', 2, 5), ('Patch-D', 4, 6), ('Patch-E', 5, 7), ('Patch-F', 6, 8)]
  Selected: ['Patch-A', 'Patch-C', 'Patch-E']
  Count:    3
  Time:     0.0091 ms

Test: All overlapping
  Windows:  [('Patch-X1', 0, 10), ('Patch-X2', 1, 9), ('Patch-X3', 2, 8)]
  Selected: ['Patch-X3']
  Count:    1
  Time:     0.0033 ms

Test: No overlaps
  Windows:  [('Patch-Y1', 0, 1), ('Patch-Y2', 2, 3), ('Patch-Y3', 4, 5), ('Patch-Y4', 6, 7)]
  Selected: ['Patch-Y1', 'Patch-Y2', 'Patch-Y3', 'Patch-Y4']
  Count:    4
  Time:     0.0030 ms

Test: Edge — empty list
  Windows:  []
  Selected: []
  Count:    0
  Time:     0.0002 ms

Test: Edge — single window
  Windows:  [('Patch-Z1', 3, 5)]
  Selected: ['Patch-Z1']
  Count:    1
  Time:     0.0008 ms

--- Benchmark ---
Size       Time (ms)
10         0.0094
100        0.1136
1000       1.5891
5000       8.9371
10000      19.8168
```

```
=====================================================
0/1 KNAPSACK — Firewall Rule Budget
=====================================================

Test: Small / Normal
  Rules:             [('Block-SSH-Brute', 3, 60), ('DPI-Malware', 4, 80), ('Rate-Limit-DNS', 2, 40), ('Geo-Block', 5, 90), ('IDS-Signature', 1, 30)]
  Capacity:          8
  Max coverage value: 170
  Time:              0.0134 ms

Test: Edge — zero capacity
  Rules:             [('Rule-A', 2, 50), ('Rule-B', 3, 70)]
  Capacity:          0
  Max coverage value: 0
  Time:              0.0015 ms

Test: Edge — single item fits exactly
  Rules:             [('Rule-Solo', 5, 100)]
  Capacity:          5
  Max coverage value: 100
  Time:              0.0012 ms

Test: Edge — all items too heavy
  Rules:             [('Heavy-1', 10, 50), ('Heavy-2', 12, 60)]
  Capacity:          5
  Max coverage value: 0
  Time:              0.0015 ms

Test: Medium
  Rules:             [('R1', 2, 30), ('R2', 3, 45), ('R3', 4, 50), ('R4', 5, 70), ('R5', 1, 20), ('R6', 3, 55), ('R7', 6, 85), ('R8', 2, 35), ('R9', 4, 60), ('R10', 7, 100)]
  Capacity:          15
  Max coverage value: 245
  Time:              0.0231 ms

--- Benchmark ---
Items      Capacity   Time (ms)
10         30         0.0457
50         150        1.2316
100        300        5.1044
200        600        21.8798
```

Observations

Merge Sort

10 -> 0.0065ms, 100 -> 0.0955ms, 1000 -> 1.2538ms, 10000 -> 15.3238ms. Each 10x increase in input size produces roughly a 12–15x increase in time. That's the signature of O(n log n).

Activity Selection

10 -> 0.0094ms, 100 -> 0.1136ms, 1000 -> 1.5891ms, 10000 -> 19.8168ms. Same ~12x per 10x increase. Consistent with the analysis that the sort suggests.

Knapsack

- 10 items, cap 30 -> 0.0457ms
- 50 items, cap 150 -> 1.2316ms (27x jump for 5x items)
- 100 items, cap 300 -> 5.1044ms (~4x jump for 2x items)
- 200 items, cap 600 -> 21.8798ms (~4x jump for 2x items)

Note: Sub-millisecond measurements appear to contain significant noise, so anomalies (like the already-sorted merge sort case running faster than theory predicts) are likely measurement artifacts rather than algorithmic behavior.

Section 6: Conclusion

In regards to the activity selector, we acknowledge that the actual situation of server maintenance is much more nuanced and that there are typically priorities associated with tasks as opposed to just “squeezing as much in as possible”. Doing it the way we implemented it could lead to large maintenance “starvation” since the algorithm always sorts by run-time and always picks the quickest tasks first until the window is filled up.

There also was a slight design miscommunication for the merge-sort, and the variable used to store the sorted list of activities needed to be re-packed to accommodate the inter-program function call.